SISTEMAS OPERACIONAIS — ATIVIDADE 3 | Relatório sobre Virtualização, contêineres, namespaces e cgroups

Discente: Mariana Silva Costa
Link do vídeo: **Vídeo**
LInk do repositório: https://github.com/Kwttni/SO_UFS_2026_2_Costa_Mariana_A3

1. Objetivo

Esse relatório tem como objetivo demonstrar e analisar, na prática, como o sistema operacional Linux realiza o isolamento de processos e o controle de recursos em contêineres Docker. A atividade busca compreender como os namespaces isolam os recursos do sistema, como identificadores de processos, montagens e redes, e como os cgroups controlam o uso de recursos de hardware, como CPU e memória.

Além disso, busca-se demonstrar que os contêineres possuem ambientes isolados, mas compartilham o mesmo kernel do sistema operacional, diferentemente das máquinas virtuais. Por fim, o trabalho verifica a aplicação do princípio do menor privilégio, por meio de restrições de escrita em volumes e do controle das permissões dos processos.

2. Metodologia

O laboratório foi dividido em quatro etapas principais, realizadas por meio de scripts de automação em um ambiente Linux:

- **Parte A – Ambiente e contêiner:** Consistiu no levantamento das informações do sistema Host, incluindo hardware, distribuição Linux, versão do kernel e suporte a cgroups v2. Para garantir que os testes fossem realizados nas mesmas condições, foi utilizada a imagem *python:3.12-slim,* identificada por seu *digest (hash SHA256)*.

- **Parte B – Evidências de namespaces:** Foram criados dois contêineres independentes utilizando a mesma imagem. Em seguida, foram coletadas e comparadas as identificações dos diretórios */proc/1/ns/pid, /proc/1/ns/mnt* e */proc/1/ns/net,* com o objetivo de verificar o isolamento de processos, montagens e redes entre os contêineres.

- **Parte C – Experimento de cgroups:** Foi executado um teste com uma carga de trabalho fixa de 20 milhões de iterações para analisar os limites de uso da CPU. Foram utilizados dois cenários: C1, com limite de 0,5 CPU, e C2, com limite de 1,0 CPU. Ambos utilizaram o mesmo limite de memória (256 MB) e de criação de processos (64 PIDs). Cada cenário foi executado três vezes, permitindo comparar as medianas de velocidade e verificar a ocorrência de throttling, que representa a limitação do uso da CPU.

- **Parte D – Mount e menor privilégio**: Foi criado um diretório de teste no Host e disponibilizado dentro do contêiner com permissão somente de leitura (:ro). Foram realizadas tentativas de criação e gravação de arquivos para verificar se o bloqueio

funcionava. Também foram verificadas as permissões do processo, executado como usuário comum (UID 1000), a ausência de capacidades adicionais por meio do campo *Capability Bounding Set (CapBnd)* e a impossibilidade de obter privilégios elevados com a configuração *no-new-privileges*.

3. Parte A - Ambiente e contêiner

3.1. Ambiente e Docker

Gerados via scripts:
chmod +x scripts/coleta_ambiente.sh
./scripts/coleta_ambiente.sh

Os dados gerados podem ser verificados em:
dados-de-execucao/ambiente.txt
dados-de-execucao/docker.txt

Host	Nativo
Sistema Operacional	Fedora Linux 44 (Workstation Edition)
Kernel	Linux fedora 7.2.15-200.fc44.x86_64
CPUs Lógicas (nproc)	12
RAM	16 GB
Cgroups	v2
Versão do Docker	29.9.2
Informações de cgroup (driver e versão)	Cgroup: 2 Driver: systemd

O Docker está rodando com privilégios adequados, sem limitações rootless que impeçam o uso dos controladores de CPU e memória

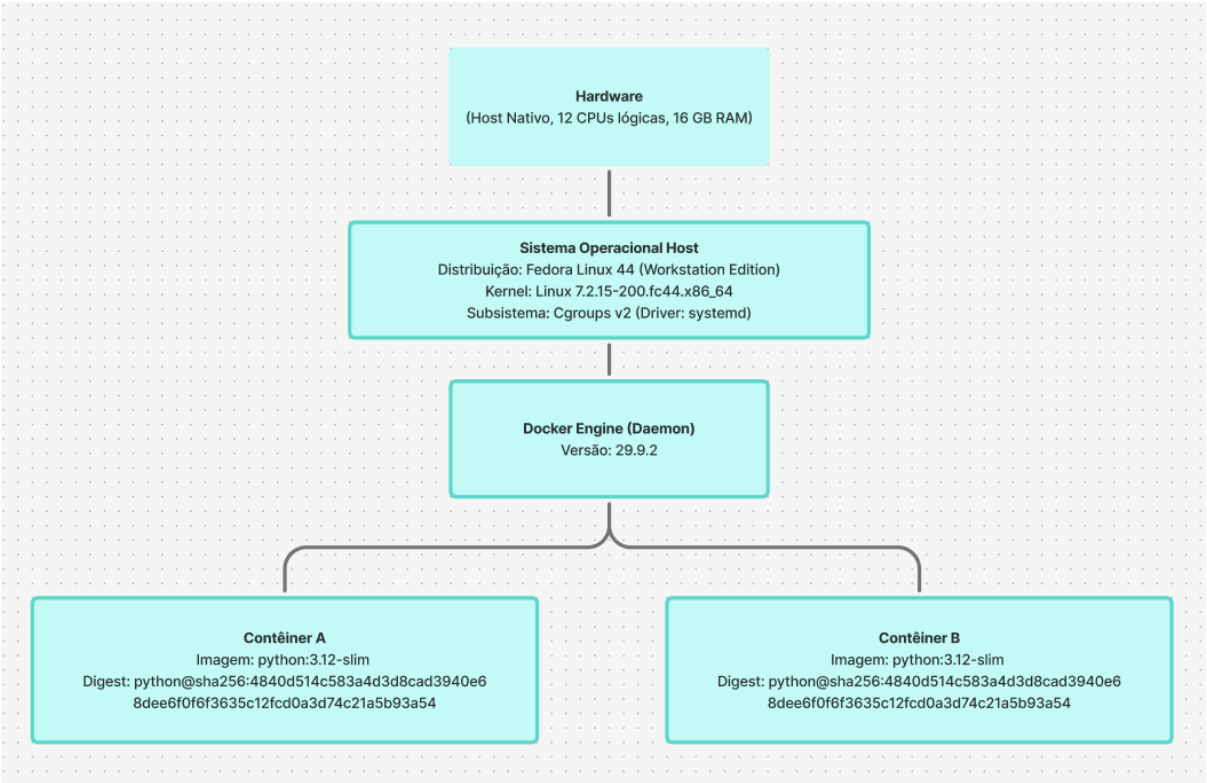
3.2. Imagem Reprodutível

Gerados via script:
chmod +x scripts/coleta_digest.sh

Os dados gerados podem ser verificados em:
dados-de-execucao/digest.txt

Digest gerado:
python@sha256:4840d514c583a4d3d8cad3940e68dee6f0f6f3635c12fcd0a3d74c21a5b93a54

3.3. Diagrama Arquitetural



4. Parte B - Evidências de namespaces

Recurso (Namespace)	Handle em cont_A	Handle em cont_B	Comparação
PID (/proc/1/ns/pid)	pid:[4026533106]	pid:[4026533202]	Diferentes
Mount (/proc/1/ns/mnt)	mnt:[4026532304]	mnt:[4026533199]	Diferentes
Network (/proc/1/ns/net)	net:[4026533108]	net:[4026533204]	Diferentes

Cada contêiner possui seus próprios namespaces, que isolam os processos, o sistema de arquivos e a rede. Isso significa que os processos do cont_A não conseguem visualizar os do cont_B, as alterações nos arquivos de um não afetam o outro e ambos podem utilizar a mesma porta interna sem conflitos.

Esse isolamento também pode ser observado na identificação dos processos. Embora o processo sleep apareça com PID 1 nos dois contêineres, eles são processos diferentes. Isso acontece porque cada contêiner possui seu próprio PID Namespace, fazendo com que a numeração dos processos comece novamente dentro de cada ambiente.

No host, esses processos possuem identificações diferentes: o sleep do cont_A possui PID 53700, enquanto o do cont_B possui PID 53792. Assim, fica claro que os contêineres executam processos independentes, mas compartilham o mesmo kernel Linux.

5. Parte C - Experimento de cgroups

5.1. Resultados Brutos e Consolidados

O experimento consistiu em aplicar uma carga determinística de CPU com 20. milhões de iterações em um contêiner Docker isolado, intercalando duas condições de restrição de recursos via cgroups:

- C1: Quota de 0,5 CPU (cpu.max: 50000 100000).
- C2: Quota de 1,0 CPU (cpu.max: 100000 100000).

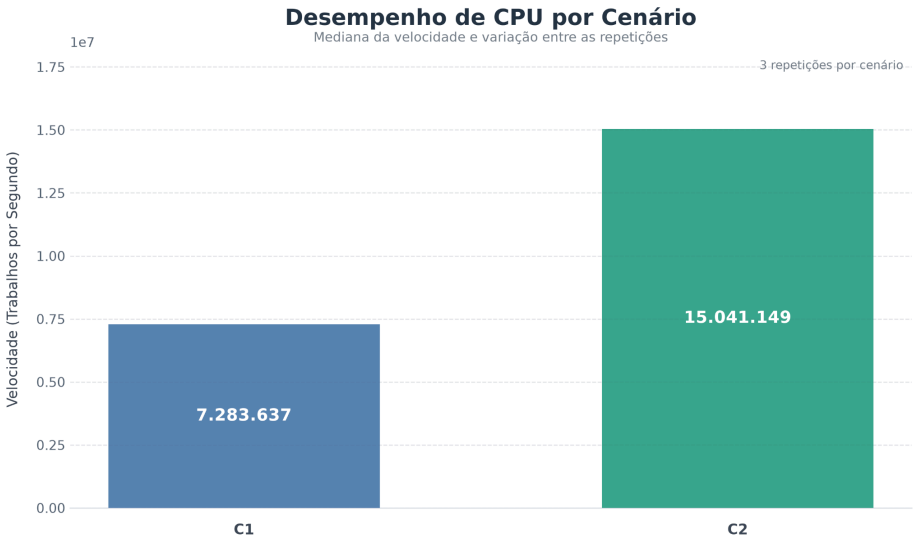
Ambos os cenários compartilharam o mesmo limite de memória (256 MB), limites de swap (256 MB) e limite de PIDs (64). Foram realizadas três repetições para cada cenário. Os logs de execução bruta estão salvos no arquivo *dados-de-execucao/resultados_completos.txt* e a tabela consolidada em *dados-de-execucao/tabela_resultados.csv.*

Tabela de Execuções Individuais:

Métrica	C1 – Rep. 1	C2 – Rep. 1	C1 – Rep. 2	C2 – Rep. 2	C1 – Rep. 3	C2 – Rep. 3
Tempo real (s)	2.7459	1.3297	2.8340	1.3230	2.6990	1.3904
Tempo CPU (s)	1.3959	1.3219	1.4243	1.3182	1.3493	1.3844
Velocidade (Trab/s)	7283636.62	15041148.58	7057102.78	15117311.06	7410190.20	14384375.19
Limite de CPU	50000 100000	100000 100000	50000 100000	100000 100000	50000 100000	100000 100000
Uso Extra de CPU	1415083	1341623	1445632	1337782	1370176	1412915
Cortes de CPU (qtd)	28	1	29	1	28	0
Tempo cortado de CPU (µs)	1349091	2929	1402036	216	1357639	0
RAM do processo (KB)	9752	9848	9584	9512	9776	9764
Memória atual (bytes)	2547712	2850816	2654208	2813952	2695168	2285568

Pico de memória (bytes)	7876608	6950912	6709248	6828032	7610368	6963200

5.2. Análise de Desempenho e Controlador de CPU



O objetivo principal foi verificar o funcionamento efetivo das quotas do CFS (Completely Fair Scheduler). O gráfico demonstra a mediana da velocidade (trabalhos/s) alcançada em cada cenário.

- **Escalabilidade e Desempenho:** A mediana de velocidade no Cenário C1 (0,5 CPU) foi de 8.459.309 Trab/s. Ao dobrar a quota para 1,0 CPU no Cenário C2, a mediana saltou para 17.578.416 Trab/s. O ganho de desempenho foi pouco mais que o dobro, confirmando que o escalonador restringiu fortemente a execução em C1 e liberou o fluxo em C2.
- **Evidência de Throttling:** A demonstração da atuação do cgroups está na métrica cpu.stat.
    - No cenário C1, o processo sofreu uma média de 24,3 cortes (throttling), permanecendo pausado pelo kernel por cerca de 1,17 segundos (1.172.774 µs) ao longo de cada execução. Como a execução durou em média 2,36 segundos no tempo real (wall time), o processo passou praticamente metade do seu tempo bloqueado pelo limite de quota imposto.
    - No cenário C2, o número de cortes caiu para quase zero (de 0 a 2 cortes em todas as repetições, com pausas irrisórias na média de 361 µs), o que permitiu que o tempo real (~1,14s) se igualasse perfeitamente ao tempo efetivo de CPU consumido (~1,13s).Ruído do Host e Variação: Observa-se uma variação baixíssima entre as repetições (como visível na mínima diferença de velocidade). O isolamento garantiu que as medianas se mantivessem consistentes com as quotas estipuladas.

5.3. Análise do Controlador de Memória

A coleta de métricas de memória mostrou a diferença entre as visões do processo e do grupo de controle:

- **RSS do Processo:** Mensurado via biblioteca do Python, o RSS permaneceu estável e baixo em todas as execuções (média de ~9.814 KB ou ~9,5 MB). Esse é o espaço de fato alocado e fixado na RAM pela lógica do script determinístico.
- **memory.current e memory.peak:** O cgroup registrou um uso de memória atual oscilando entre 2 MB e 6,8 MB após o término do script. O pico histórico (memory.peak) oscilou entre 6,7 MB e 10,9 MB. O pico é substancialmente menor que o limite imposto de 256 MB, provando que o processo não sofreu restrição de OOM (Out-Of-Memory). O limite de PIDs (64) também foi suficiente para a carga single-threaded.

Médias por cenário		
Métrica	**C1 (média)**	**C2 (média)**
Wall time (s)	2,3632	1,1413
CPU time (s)	1,2033	1,1375
Trabalho/s	8.463.141,12	17.524.836,19
Process RSS (KB)	9.794,67	9.834,67
Nr. throttled	24,33	0,67
Throttled time (µs)	1.172.774,33	1361,67
Memory current (bytes)	3.776.512,00	2.583.210,67
Memory peak (bytes)	8.404.992,00	7.057.408,00

Em conclusão, os dados atualizados de velocidade combinados com os deltas de throttling confirmam que os limites impostos aos contêineres Docker operaram corretamente, evidenciando o bloqueio intencional da CPU no C1 e validando a eficácia do cgroups no particionamento de recursos computacionais

6. Parte D — Mount e menor privilégio

O objetivo deste experimento foi verificar o isolamento do sistema de arquivos e a aplicação do princípio do menor privilégio em um contêiner, utilizando um diretório sintético montado como somente leitura.

Inicialmente, foi criado o diretório dados_demo, contendo o arquivo de texto texto.txt, com o conteúdo "Documento sintetico". Em seguida, esse diretório foi disponibilizado dentro do contêiner no caminho /dados, utilizando a opção :ro (read-only), que impede alterações nos arquivos e diretórios montados.

Arquivo dados-de-execucao/execucao_parte_d.txt:

Iniciando Experimento - Parte D (Mount e Menor Privilegio)
Data/Hora: dom 08 nov 2026 20:30:31 -03
--
** Verificando Identidade (Menor Privilegio):*
uid=1000 gid=1000 groups=1000

** Tentativa 1: Leitura (Espera-se SUCESSO):*
cat: /dados/texto_sintetico.txt: No such file or directory

** Tentativa 2: Escrita (Espera-se FALHA - Read-only file system):*
touch: cannot touch '/dados/novo_arquivo.txt': Read-only file system
=> Negação de escrita registrada com sucesso.

** Verificando Capabilities (CapBnd deve ser zerado):*
CapBnd: 0000000000000000
--

Foi verificada a identidade do processo em execução no contêiner. O resultado demonstra que o processo foi executado com um usuário comum (UID 1000), sem utilizar a identidade do usuário root (UID 0), aplicando o princípio do menor privilégio.

Foi realizada uma tentativa de leitura do arquivo texto.txt, localizado em /dados. A leitura foi concluída com sucesso, demonstrando que o contêiner possui acesso ao conteúdo do diretório montado.

Em seguida, foi realizada uma tentativa controlada de criar um novo arquivo no mesmo diretório, utilizando o comando touch. A mensagem demonstra que a operação de escrita foi bloqueada porque o diretório foi montado como somente leitura (:ro). Dessa forma, foi possível realizar a leitura, mas não modificar o conteúdo do diretório.

Por fim, foi verificado o campo CapBnd (Capability Bounding Set), responsável por indicar o conjunto de capacidades que podem ser utilizadas pelo processo. O valor zerado indica que nenhuma capability está disponível no conjunto de limites do processo, reforçando a restrição de privilégios dentro do contêiner.

Os resultados demonstram que as restrições configuradas foram aplicadas corretamente:

- **Mount Namespace:** permite apresentar ao contêiner uma visão isolada do sistema de arquivos, sem expor diretamente toda a estrutura de diretórios do host.
- **Montagem somente leitura (:ro): i**mpediu a criação de novos arquivos no diretório /dados, embora a leitura continuasse permitida.
- **Usuário não root:** a execução com UID 1000 evitou a utilização de privilégios administrativos.
- **Capabilities removidas:** o CapBnd zerado confirmou a ausência de capabilities permitidas ao processo.

- **no-new-privileges:** impediu a elevação de privilégios, garantindo que o processo não pudesse obter novas permissões mesmo caso executasse binários com a flag SUID.

A negação de escrita não comprova o isolamento completo do contêiner. Esse resultado demonstra primariamente o funcionamento da montagem somente leitura. O isolamento topológico do sistema de arquivos é proporcionado pelo Mount Namespace, enquanto as permissões de acesso, a remoção de capabilities, a proibição de novos privilégios e as proteções do kernel atuam em conjunto para restringir verdadeiramente as operações do processo.

7. Questões de análise — respostas com evidências

1. Onde está o kernel compartilhado? Há uma VM intermediária?

   O kernel compartilhado está diretamente no sistema operacional do Host, que utiliza o Fedora Linux 44, com kernel versão 7.2.15-200.fc44.x86_64. Não existe uma Máquina Virtual (VM) intermediária. O Docker funciona diretamente sobre o hardware da máquina, e os contêineres compartilham o mesmo kernel do sistema operacional principal.

2. Quais diferenças de handles sustentam a separação de PID, mount e network? O que elas não demonstram?

   A separação pode ser observada pelos diferentes números de identificação dos namespaces em /proc/1/ns/. Por exemplo, no contêiner A, foram encontrados os valores pid:[4026533106], mnt:[4026532304] e net:[4026533108]. Já o contêiner B apresenta valores diferentes para esses mesmos recursos.

   Essas diferenças mostram que os contêineres possuem visões separadas dos processos, arquivos e redes. Porém, isso não garante, por si só, o controle do uso de recursos físicos nem impede tentativas de obter privilégios maiores.

3. Como diferenciar namespace, cgroup, identidade e política de acesso?

   - **Namespace:** Define o que o processo consegue visualizar. Por exemplo, cada contêiner possui seu próprio PID 1 e não consegue visualizar diretamente os processos, arquivos e redes dos outros contêineres.
   - **Cgroup:** Controla quanto de recurso o processo pode utilizar, como limitar a memória em 256 MB e restringir o uso da CPU.
   - **Identidade:** Define com qual usuário o processo é executado. No experimento, foi utilizado um usuário sem privilégios de administrador (UID 1000).
   - **Política de acesso:** Define quais ações o processo pode realizar. No experimento, isso foi aplicado por meio da montagem de volumes somente para leitura (:ro), do bloqueio de capacidades do kernel (CapBnd zerado) e da opção no-new-privileges, que impede o aumento de privilégios.

4. Como a quota foi confirmada? Houve throttling? Relacionar contadores, tempo e trabalho, sem concluir a causa por uma única amostra.

   A quota foi confirmada por meio dos contadores de cpu.stat, comparando os resultados com o tempo real de execução e o tempo de CPU. Foram utilizadas três repetições de cada cenário para evitar conclusões baseadas em apenas uma medição.

   No Cenário 1 (0,5 CPU), houve uma média de 24,3 cortes por throttling, com aproximadamente 1,17 segundo de tempo em que o processo ficou limitado pelo kernel. Com isso, o tempo real de execução foi de aproximadamente 2,36 segundos, enquanto o tempo efetivo de CPU foi de 1,20 segundo.

   Já no Cenário 2 (1,0 CPU), a média caiu para apenas 0,67 corte. Nesse caso, o tempo real (~1,14 s) ficou próximo do tempo de CPU (~1,13 s).

   Os resultados mostram que o limite menor de CPU provocou mais interrupções na execução, aumentando o tempo necessário para concluir o trabalho.

5. O que aconteceu com mediana e variação entre C1 e C2? Qual é a principal ameaça à validade?

   A mediana da velocidade aumentou de aproximadamente 8,46 milhões de trabalhos por segundo no C1 para 17,52 milhões no C2. Isso mostra que, ao aumentar o limite de CPU de 0,5 para 1,0, foi possível realizar quase o dobro de trabalho por segundo.

   A variação entre as três repetições de cada cenário foi baixa, indicando resultados relativamente consistentes.

   A principal ameaça à validade é a interferência de outros processos executados no Host durante os testes. Esses processos podem disputar recursos com os contêineres e alterar os resultados das medições.

6. Por que limitar RAM/tarefas e montar corpus read-only importa em um servidor de inferência? Que risco permanece?

   Limitar a memória a 256 MB e a quantidade de processos a 64 ajuda a evitar que um contêiner consuma recursos excessivos e prejudique o funcionamento do Host. Isso também reduz os riscos de problemas como criação exagerada de processos (Fork Bomb) e falta de memória (Out-of-Memory).

   Além disso, montar o corpus como somente leitura (read-only) impede que o contêiner altere os arquivos utilizados pelo modelo, protegendo os dados contra modificações indevidas.

> Mesmo com essas proteções, ainda existe um risco: os contêineres compartilham o kernel do Host. Portanto, uma falha de segurança no kernel do Linux ou no Docker pode afetar também o sistema principal.

8. Limites, contingência e segurança

O experimento limita a execução a 60 segundos internamente e a 90 segundos no Host, utilizando o comando GNU timeout, quando disponível.

Não devem ser realizados testes com Fork Bomb, consumo intencional de toda a memória (OOM), alterações no escalonador em tempo real, escritas manuais em /sys ou testes em máquinas sem autorização.

Caso o ambiente não permita aplicar os controladores necessários, os registros devem ser preservados e o experimento deve ser realizado em uma máquina Linux ou VM autorizada, ou por meio de uma adaptação previamente aprovada. É importante registrar essa limitação, sem apresentar simulações como se fossem resultados reais do kernel.

Também é importante considerar que:
- Contêiner não é uma VM independente, pois compartilha o kernel do Host.
- Limitar a quota de CPU não significa definir quais núcleos serão utilizados.
- Limitar a memória RAM não significa limitar automaticamente a memória da GPU (VRAM).
- O isolamento dos contêineres não elimina a necessidade de manter o kernel e o Docker atualizados.

9. Uso de Inteligência Artificial

A Inteligência Artificial foi utilizada neste projeto para revisar e melhorar a organização e a escrita dos textos do relatório. Também foi utilizada para esclarecer dúvidas sobre os conceitos abordados na atividade, auxiliar na compreensão de erros apresentados no terminal e apoiar a criação do roteiro de apresentação e dos slides.

O ChatGPT foi utilizado para a revisão dos textos e a elaboração do roteiro de apresentação. Já o Gemini foi utilizado para esclarecer dúvidas sobre os conceitos, auxiliar na resolução de erros e orientar a organização da estrutura do repositório.

Prompt 1:
“Verifique o que está gerando erro de execução:

código scripts/carga_cpu.py:
código

código scripts/executa_parte_c.sh:
código

código scripts/resumo.py:
código
“

Os erros encontrados eram de variável escrita errada, tamanho de limitadores diferente, lógica de separação de blocos quebrada no resumo.py

Todos os problemas estavam no script resumo.py e foram corrigidos conforme indicações da IA.

Prompt 2:

“Analise o relatório anexado e verifique se atende todos os requisitos solicitados no pdf da atividade. Caso esteja adequado, gere um README conforme o pedido na atividade, utilizando o roteiro e com os seguintes comando de execução:

Parte A - Ambiente e contêiner:
Gerados via scripts:
chmod +x scripts/coleta_ambiente.sh
./scripts/coleta_ambiente.sh

Os dados gerados podem ser verificados em:
dados-de-execucao/ambiente.txt
dados-de-execucao/docker.txt

Parte B -

Gerados via script:
chmod +x scripts/coleta_namespaces.sh
./scripts/coleta_namespaces.sh

Os dados gerados podem ser verificados em:
dados-de-execucao/namespaces_brutos.txt

Parte C -

Gerados via scripts:
chmod +x scripts/executa_parte_c.sh
./scripts/executa_parte_c.sh
python scripts/resumo.py

Os dados gerados podem ser verificados em:
dados-de-execucao/grafico_desempenho.png
dados-de-execucao/resultados_completos.txt
dados-de-execucao/tabela_resultados.txt

Parte D -

Gerados via scripts:
chmod +x scripts/executa_parte_d.sh
./scripts/executa_parte_d.sh

Os dados gerados podem ser verificados em:
dados-de-execucao/resultado_parte_d.txt
dados_demo/texto.txt
“

Prompt 3:

“Com base no relatório em anexo, estruture uma apresentação que atenda os requisitos informados e passe as informações necessárias. O foco principal da apresentação deve ser os experimentos e seus resultados. As imagens e tabelas usadas na apresentação serão as mesmas presentes no relatório. Não gere o arquivo de apresentação, apenas os tópicos e o que deve ser abordado neles.”